

Agentic Sales Orchestrator

Journey Participation Ledger Programmatic Runbook - True Line-by-Line Edition

Item	Value
Environment	Phoenicarix-CI - https://phoenicarix-ci.crm4.dynamics.com
Solution	ASO.Core - unique name ASOCore
Target table	Journey Participation Ledger - logical name aso_journeyparticipationledger
Implementation method	.NET 8 console app using Microsoft.PowerPlatform.Dataaverse.Client
Date / version	2026-05-24 / v1.0
Scope	Create custom ledger table and required ASO journey participation fields. No journeys are activated; no emails are sent.

1. Executive summary

We created the custom Dataverse table Journey Participation Ledger programmatically inside ASO.Core in the Phoenicarix-CI environment. The table prepares the Agentic Sales Orchestrator MVP for future Customer Insights journey tracking by storing participation status, journey references, entry source, timestamps, interaction type, exit reason, correlation ID, and error information. In simple words: we added a controlled audit notebook for future customer-journey activity; we did not start a journey and did not send communication.

2. Business objective

- Provide a governed place to record which sales-related records participate in Customer Insights journeys.
- Support auditability for future lifecycle communication states without mixing technical trace data into standard sales fields only.
- Allow Product Owners and operations teams to understand why a lead, opportunity, account, or contact entered, stayed in, or exited a journey.
- Prepare later Power Automate / Customer Insights writeback logic without enabling customer communication yet.

3. Technical objective

The technical objective was to create the custom table aso_journeyparticipationledger and the required metadata columns through the Dataverse SDK. The script uses CreateEntityRequest for the table, CreateAttributeRequest for the fields, global choices for shared ASO lifecycle/participation statuses, local choices for table-specific values, and PublishXmlRequest to publish the result.

4. Scope

In scope	Out of scope
Create aso_journeyparticipationledger table	No Customer Insights journey activation
Create fields/columns listed in the Phase 2 implementation pattern	No outbound email or SMS
Reuse global ASO choices where appropriate	No Power Automate flow execution
Create local choices for entry source and last interaction type	No SAP or HubSpot connection
Publish table customizations and validate in ASO.Core	No model-driven form layout work in this step

5. Prerequisites

Prerequisite	Required / expected value
Environment	Phoenicarix-CI
Dataverse URL	https://phoenicarix-ci.crm4.dynamics.com
Solution	ASO.Core
Solution unique name	ASOCore
Publisher prefix	aso
Local machine	macOS with .NET 8 SDK
SDK package	Microsoft.PowerPlatform.Dataverse.Client
Security/permissions	System Administrator, System Customizer, or equivalent rights to create custom tables/columns and publish customizations
Prior ASO setup	Global choices already created in ASO.Core: aso_lifecyclecommunicationstage and aso_journeyparticipationstatus
Backup	Managed and unmanaged ASO.Core backup should exist before schema changes

6. Architecture/context

The table belongs to the ASO.Core schema layer. It is not an automation table and it does not communicate with customers directly. It supports a future integration path where Customer Insights journey entry/interaction data can be written back to Dataverse in a governed, traceable way. The implementation intentionally stores Record Type and Record ID as text fields rather than creating hard lookups to Lead, Opportunity, Account, and Contact in this MVP step. That keeps the ledger simple and avoids creating multiple relationships before the writeback contract is finalized.

7. Field inventory

Display name	Schema/logical name	Type	Size/values	Business purpose
Record Type	aso_recordtype	Text	100	Identifies the business/Dynamics record type such as Lead, Opportunity, Account, or Contact.
Record ID	aso_recordid	Text	100	Stores the Dataverse row identifier for the related business record without creating a hard lookup.
Lifecycle Communication Stage	aso_lifecyclecommunicationstage	Global Choice	aso_lifecyclecommunicationstage	Keeps journey participation aligned to the global ASO lifecycle taxonomy.
Customer Insights Journey ID	aso_customerinsightsjourneyid	Text	100	Stores the Customer Insights journey identifier.
Customer Insights Journey Name	aso_customerinsightsjourneyname	Text	200	Readable journey name for product owners, operators, and sellers.
Participation Status	aso_participationstatus	Global Choice	aso_journeyparticipationstatus	Uses the global ASO journey participation status values.
Entry Trigger Name	aso_entrytriggername	Text	200	Stores the entry trigger, segment, custom action, or operations process name.
Entry Source	aso_entrysource	Local Choice	DataverseTrigger, Segment, CustomAction, ManualOps	Classifies how the journey entry was initiated or will be explained later.
Started On	aso_startedon	Date and Time	User local	When journey participation started.
Last Interaction On	aso_lastinteractionon	Date and Time	User local	Latest known Customer Insights interaction timestamp.
Last Interaction Type	aso_lastinteractiontype	Local Choice	EmailSent, Open, Click, FormSubmit, Reply, Unsubscribe, Bounce, CustomAction	Classification of latest interaction.
Exit Reason	aso_exitreason	Text	500	Reason for journey exit/removal.
Correlation ID	aso_correlationid	Text	100	Cross-system traceability ID.
Error Message	aso_errormessage	Multiline Text	4000	Failure details if processing fails.

8. Step-by-step implementation summary

1. Created a local .NET 8 console project in ~/aso-dataverse-tools/journey-participation-ledger.
2. Added Microsoft.PowerPlatform.Dataverse.Client package.

3. Replaced Program.cs with the Dataverse metadata creation script.
4. Ran dotnet run from the folder.
5. Authenticated interactively to Phoenicarix-CI if prompted.
6. Script connected to Dataverse and printed the target environment, solution, and table.
7. Script checked whether aso_journeyparticipationledger already existed.
8. Script created the table if missing.
9. Script retrieved existing columns and created missing columns only.
10. Script published the Journey Participation Ledger customizations.
11. Validation was performed in Power Apps under ASO.Core -> Tables -> Journey Participation Ledger -> Columns.

9. Validation checklist

Check	Where	Expected result
Table exists	ASO.Core -> Tables	Journey Participation Ledger is visible.
Columns exist	ASO.Core -> Tables -> Journey Participation Ledger -> Columns	Record Type, Record ID, journey fields, status, entry fields, interaction fields, correlation and error fields exist.
Global choices reused	Open Lifecycle Communication Stage and Participation Status columns	They reference existing ASO global choices.
Local choices created	Open Entry Source and Last Interaction Type	Values match the implementation guide.
Solution association	Open ASO.Core objects/components	Table and columns are part of ASO.Core.
No communication activated	Customer Insights / Power Automate	No journey, trigger, email, or flow was activated by this step.

10. Troubleshooting guide

Symptom	Likely cause	Resolution
Connection failed	Wrong URL, expired login, missing permissions	Authenticate again and confirm the URL is Phoenicarix-CI.
Global choice creation fails	Global choice name not found or not in solution	Open ASO.Core -> Choices and confirm exact logical names.
Language code error	The label language code is not enabled/valid for the organization	Use the organization base/enabled language code. In this run, 1031 was used because the environment UI is German-oriented.
Column already exists	Script rerun or partial prior run	Expected; script skips existing columns safely.
Table visible but fields not visible	Publishing delay or maker UI cache	Refresh maker portal and publish customizations.
Components not in ASO.Core	Wrong SolutionUniqueName	Correct SolutionUniqueName to ASOCORE or add existing components to ASO.Core.

11. Risks and mitigations

Risk	Mitigation
Wrong environment targeted	Print DataverseUrl in terminal; validate pac auth and browser environment before running.
Wrong solution layer	Use SolutionUniqueName = ASOCORE and validate components inside ASO.Core.
Unwanted required fields block users	All non-primary fields are optional in this MVP.
Future relationship requirements change	Record Type/Record ID pattern avoids premature hard lookups; add relationships later when contract is stable.
Language-label issues	Use enabled language code and document the chosen value.

12. Rollback / recovery approach

For trial/MVP work, the safest recovery is restoring from managed/unmanaged ASO.Core backups or deleting the newly created custom table before dependencies are added. Once automation, forms, views, relationships, or data depend on this table, deletion becomes harder and must be assessed component-by-component. Always export managed and unmanaged backups before and after schema steps.

13. Handover notes for IT

- Keep Program.cs and exported solutions in source control or a controlled project folder.
- Record environment URL, solution unique name, run date, and terminal output screenshots in the implementation workbook.
- Do not manually rename schema/logical names after creation.
- Do not use this table as a sending mechanism; it is a ledger/audit structure.
- Later phases should define who writes to the ledger and which security roles can read or maintain it.

14. Detailed line-by-line script explanation

Line	Code	Explanation in simple language	Technical explanation	Why it matters	Common mistake / warning
1	using Microsoft.Crm.Sdk.Messages;	Imports a .NET / Dataverse namespace.	Makes SDK classes such as ServiceClient, CreateAttributeRequest, EntityMetadata, and AttributeMetadata available without fully qualified names.	Without imports, the script would not compile unless every SDK type was fully qualified.	Missing namespace imports cause compiler errors such as type or namespace not found.
2	using Microsoft.PowerPlatform.Dataverse.Client;	Imports a .NET / Dataverse namespace.	Makes SDK classes such as ServiceClient, CreateAttributeRequest, EntityMetadata, and AttributeMetadata available without fully qualified names.	Without imports, the script would not compile unless every SDK type was fully qualified.	Missing namespace imports cause compiler errors such as type or namespace not found.
3	using Microsoft.Xrm.Sdk;	Imports a .NET / Dataverse namespace.	Makes SDK classes such as ServiceClient, CreateAttributeRequest, EntityMetadata, and AttributeMetadata available without fully qualified names.	Without imports, the script would not compile unless every SDK type was fully qualified.	Missing namespace imports cause compiler errors such as type or namespace not found.
4	using Microsoft.Xrm.Sdk.Messages;	Imports a .NET / Dataverse namespace.	Makes SDK classes such as ServiceClient, CreateAttributeRequest, EntityMetadata, and AttributeMetadata available without fully qualified names.	Without imports, the script would not compile unless every SDK type was fully qualified.	Missing namespace imports cause compiler errors such as type or namespace not found.
5	using Microsoft.Xrm.Sdk.Metadata;	Imports a .NET / Dataverse namespace.	Makes SDK classes such as ServiceClient, CreateAttributeRequest, EntityMetadata, and AttributeMetadata available without fully qualified names.	Without imports, the script would not compile unless every SDK type was fully qualified.	Missing namespace imports cause compiler errors such as type or namespace not found.
6	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
7	const string DataverseUrl = "https://phoenixarix-ci.crm4.dynamics.com";	Defines the target Dataverse environment URL.	All SDK calls are sent to this environment: Phoenixarix-CI.	Prevents creating schema in the wrong trial environment.	Wrong URL creates or changes components in the wrong environment.
8	const string SolutionUniqueName = "ASOCore";	Defines the target solution unique name.	The SDK uses this value in metadata requests to associate created components with ASO.Core.	Keeps schema in the correct ALM solution layer.	Wrong value can place fields outside ASO.Core or fail the request.
9	const string EntityLogicalName = "aso_journeyparticipationledger";	Defines the custom table logical name.	The script targets aso_journeyparticipationledger.	Ensures the script creates metadata on the intended Journey Participation Ledger table.	Wrong logical name causes retrieval or creation against the wrong table or a not-found error.
10	const int LanguageCode = 1031;	Defines the language code used for labels.	1031 is German and was used because the Phoenixarix-CI organization uses German UI/base language behavior for some metadata labels.	Labels and descriptions must use a language enabled in the organization.	Invalid language code errors can occur if 1033/1031 does not match enabled languages.
11	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
12	var connectionString =	Builds the OAuth connection string.	ServiceClient reads this string to authenticate interactively and connect to Dataverse.	Needed before any table or column metadata can be read or created.	Wrong client ID, URL, redirect URI, or authentication mode prevents login.
13	\$@"AuthType=OAuth;	Builds the OAuth connection string.	ServiceClient reads this string to authenticate interactively and connect to Dataverse.	Needed before any table or column metadata can be read or created.	Wrong client ID, URL, redirect URI, or authentication mode prevents login.
14	Url={DataverseUrl};	Builds the OAuth connection string.	ServiceClient reads this string to authenticate interactively and connect to Dataverse.	Needed before any table or column metadata can be read or created.	Wrong client ID, URL, redirect URI, or authentication mode prevents login.
15	LoginPrompt=Auto;	Builds the OAuth connection string.	ServiceClient reads this string to authenticate interactively and connect to Dataverse.	Needed before any table or column metadata can be read or created.	Wrong client ID, URL, redirect URI, or authentication mode prevents login.
16	ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;	Builds the OAuth connection string.	ServiceClient reads this string to authenticate interactively and connect to Dataverse.	Needed before any table or column metadata can be read or created.	Wrong client ID, URL, redirect URI, or authentication mode prevents login.
17	RedirectUri=http://localhost";	Builds the OAuth connection string.	ServiceClient reads this string to authenticate interactively and connect to Dataverse.	Needed before any table or column metadata can be read or created.	Wrong client ID, URL, redirect URI, or authentication mode prevents login.
18	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
19	using var service = new ServiceClient(connectionString);	Imports a .NET / Dataverse namespace.	Makes SDK classes such as ServiceClient, CreateAttributeRequest, EntityMetadata, and AttributeMetadata available without fully qualified names.	Without imports, the script would not compile unless every SDK type was fully qualified.	Missing namespace imports cause compiler errors such as type or namespace not found.
20	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
21	if (!service.IsReady)	Checks connection readiness.	The script verifies authentication/connectivity before schema creation.	Prevents dangerous or confusing failures after partial setup.	Skipping this can hide connection problems until later.

22	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
23	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output color.	Red/yellow/green output makes failures, skips, and success visible.	Improves operator confidence during execution.	Not functionally required, but helps avoid missing errors.
24	Console.WriteLine("Connection failed.");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
25	Console.WriteLine(service.LastError);	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
26	Console.ResetColor();	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
27	return;	Stops execution early.	Used when the connection is not ready.	Prevents schema changes when the script cannot safely connect.	If removed, later lines may throw confusing exceptions.
28	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
29	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
30	Console.WriteLine(\$"Connected to: {DataverseUri}");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
31	Console.WriteLine(\$"Target solution: {SolutionUniqueName}");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
32	Console.WriteLine(\$"Target table: {EntityLogicalName}");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
33	Console.WriteLine();	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
34	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
35	if (!EntityExists(service, EntityLogicalName))	Checks whether the custom table already exists.	Allows the script to create the table only once and skip it safely on rerun.	Makes the script partially idempotent.	Without this check, rerunning could fail on duplicate table creation.
36	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
37	Console.WriteLine("Creating Journey Participation Ledger table...");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
38	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
39	var createEntityRequest = new CreateEntityRequest	Creates a table-creation request.	The SDK sends EntityMetadata and primary attribute metadata to Dataverse.	This is what creates the custom Journey Participation Ledger table.	Missing or wrong metadata can create the wrong table or fail validation.
40	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
41	Entity = new EntityMetadata	Defines table metadata.	Specifies schema name, display labels, description, and ownership behavior.	Controls how the new custom table appears and behaves in Dataverse.	Wrong ownership or schema name is difficult to change later.
42	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
43	SchemaName = "aso_journeyparticipationledger",	Sets a schema name.	Dataverse uses schema/logical names for ALM, SDK, API, integrations, and automation.	The aso_ naming keeps ASO components clearly separated.	Typos become permanent technical debt.
44	DisplayName = Label("Journey Participation Ledger"),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
45	DisplayCollectionName = Label("Journey Participation Ledgers")	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
46	Description = Label("Records journey participation and interaction state once the Customer Insights phase is implemented."),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
47	OwnershipType = OwnershipTypes.OrganizationOwned	Sets table ownership model.	OrganizationOwned means records are not owned by individual users; they behave as shared operational/audit records.	Appropriate for ledger-style audit records.	Wrong ownership model can complicate security and sharing.
48	},	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
49	PrimaryAttribute = new StringAttributeMetadata	Defines the primary name column.	Every custom Dataverse table needs a primary name attribute.	Allows rows to appear with readable names in views and lookups.	Missing or poor naming makes records hard to identify.
50	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
51	SchemaName = "aso_name",	Sets a schema name.	Dataverse uses schema/logical names for ALM, SDK, API, integrations, and automation.	The aso_ naming keeps ASO components clearly separated.	Typos become permanent technical debt.
52	DisplayName = Label("Name"),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
53	Description = Label("Primary name for the Journey Participation Ledger record."),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
54	RequiredLevel = new AttributeRequiredLevelManagedProperty(Attribut eRequiredLevel.ApplicationRequired),	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
55	MaxLength = 200	Sets maximum text length.	Dataverse requires length definitions for text/memo fields.	Prevents uncontrolled storage and aligns to expected payload size.	Too short truncates data; too long can be unnecessary.
56	},	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
57	SolutionUniqueName = SolutionUniqueName	Associates created metadata with ASO.Core.	Dataverse stores the column as part of the target unmanaged solution.	Important for export, backup, managed deployment, and ALM.	Wrong solution name can create solution-layer confusion.
58	};	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
59	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
60	service.Execute(createEntityRequest);	Executes the table creation request.	This sends the CreateEntityRequest to Dataverse.	The table is created at this point if it did not already exist.	If it fails, no custom table is created.
61	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
62	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output color.	Red/yellow/green output makes failures, skips, and success visible.	Improves operator confidence during execution.	Not functionally required, but helps avoid missing errors.
63	Console.WriteLine("CREATED TABLE: aso_journeyparticipationledger");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
64	Console.ResetColor();	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
65	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.

66	else	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
67	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
68	Console.ForegroundColor = ConsoleColor.Yellow;	Changes terminal output color.	Red/yellow/green output makes failures, skips, and success visible.	Improves operator confidence during execution.	Not functionally required, but helps avoid missing errors.
69	Console.WriteLine("SKIP existing table: aso_journeyparticipationledger");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
70	Console.ResetColor();	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
71	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
72	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
73	var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);	Retrieves existing column logical names.	The script reads table metadata before creating columns.	Allows safe reruns by skipping existing columns.	Without this, duplicate columns cause errors.
74	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
75	var fields = new List<AttributeMetadata>	Starts the field inventory list.	Each list entry is an AttributeMetadata definition for one required column.	Converts the implementation guide into executable metadata instructions.	Missing fields from this list will not be created.
76	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
77	Text("aso_recordtype", "Record Type", 100, "Business or Dataverse record type, for example Lead, Opportunity, Account, or Contact.");	Defines a text column to create.	The helper returns StringAttributeMetadata with schema name, label, description, required level, and max length.	Text fields store IDs, names, correlation references, and short explanations.	Wrong length or schema name can limit data or break integration mappings.
78	Text("aso_recordid", "Record ID", 100, "Dataverse row identifier of the related business record.");	Defines a text column to create.	The helper returns StringAttributeMetadata with schema name, label, description, required level, and max length.	Text fields store IDs, names, correlation references, and short explanations.	Wrong length or schema name can limit data or break integration mappings.
79	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
80	GlobalChoice("aso_lifecyclecommunicationstage", "Lifecycle Communication Stage", "aso_lifecyclecommunicationstage", "Lifecycle communication stage reused from the global ASO choice.");	Defines a choice column that reuses an existing global choice.	The helper builds PicklistAttributeMetadata with IsGlobal = true and the existing option set name.	Keeps ASO lifecycle and participation values consistent across tables.	Wrong global choice name causes metadata creation failure.
81	Text("aso_customerinsightsjourneyid", "Customer Insights Journey ID", 100, "Customer Insights journey identifier.");	Defines a text column to create.	The helper returns StringAttributeMetadata with schema name, label, description, required level, and max length.	Text fields store IDs, names, correlation references, and short explanations.	Wrong length or schema name can limit data or break integration mappings.
82	Text("aso_customerinsightsjourneyname", "Customer Insights Journey Name", 200, "Customer Insights journey name.");	Defines a text column to create.	The helper returns StringAttributeMetadata with schema name, label, description, required level, and max length.	Text fields store IDs, names, correlation references, and short explanations.	Wrong length or schema name can limit data or break integration mappings.
83	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
84	GlobalChoice("aso_participationstatus", "Participation Status", "aso_journeyparticipationstatus", "Journey participation status reused from the global ASO choice.");	Defines a choice column that reuses an existing global choice.	The helper builds PicklistAttributeMetadata with IsGlobal = true and the existing option set name.	Keeps ASO lifecycle and participation values consistent across tables.	Wrong global choice name causes metadata creation failure.
85	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
86	Text("aso_entrytriggername", "Entry Trigger Name", 200, "Name of the journey entry trigger, segment, custom action, or manual operation.");	Defines a text column to create.	The helper returns StringAttributeMetadata with schema name, label, description, required level, and max length.	Text fields store IDs, names, correlation references, and short explanations.	Wrong length or schema name can limit data or break integration mappings.
87	LocalChoice("aso_entrysource", "Entry Source",	Defines a local choice column.	The helper builds a table-specific option set for values that are not reused globally.	Useful for Entry Source and Last Interaction Type values that are specific to this table's context.	Wrong values create misleading filtering/reporting choices.
88	new[] { "DataverseTrigger", "Segment", "CustomAction", "ManualOps" },	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
89	"Source that caused or will later explain the journey entry.");	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
90	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
91	DateTimeField("aso_startedon", "Started On", "Timestamp when the journey participation started.");	Defines a date/time column.	The helper returns DateTimeAttributeMetadata using DateAndTime and UserLocal behavior.	Needed for journey start and interaction timestamps.	Wrong behavior can show unexpected times to users in different time zones.
92	DateTimeField("aso_lastinteractionon", "Last Interaction On", "Timestamp of the latest known interaction.");	Defines a date/time column.	The helper returns DateTimeAttributeMetadata using DateAndTime and UserLocal behavior.	Needed for journey start and interaction timestamps.	Wrong behavior can show unexpected times to users in different time zones.
93	LocalChoice("aso_lastinteractiontype", "Last Interaction Type",	Defines a local choice column.	The helper builds a table-specific option set for values that are not reused globally.	Useful for Entry Source and Last Interaction Type values that are specific to this table's context.	Wrong values create misleading filtering/reporting choices.
94	new[] { "EmailSent", "Open", "Click", "FormSubmit", "Reply", "Unsubscribe", "Bounce", "CustomAction" },	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
95	"Latest tracked Customer Insights interaction type.");	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
96	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
97	Text("aso_extreason", "Exit Reason", 500, "Reason why the record exited or was removed from the journey.");	Defines a text column to create.	The helper returns StringAttributeMetadata with schema name, label, description, required level, and max length.	Text fields store IDs, names, correlation references, and short explanations.	Wrong length or schema name can limit data or break integration mappings.
98	Text("aso_correlationid", "Correlation ID", 100, "Correlation ID shared across Dataverse, Power Automate, Customer Insights, and orchestration components.");	Defines a text column to create.	The helper returns StringAttributeMetadata with schema name, label, description, required level, and max length.	Text fields store IDs, names, correlation references, and short explanations.	Wrong length or schema name can limit data or break integration mappings.
99	Memo("aso_errormessage", "Error Message", "Error details if journey participation processing fails.");	Defines a multiline text column.	The helper returns MemoAttributeMetadata with MaxLength 4000.	Needed for error messages or longer operational explanations.	Using short text for long error text can truncate important information.
100	};	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
101	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.

			sections.		read.
102	foreach (var field in fields)	Starts the creation loop.	The script iterates through every planned field metadata definition.	Automates repetitive column creation safely.	Loop errors can stop or skip field creation.
103	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
104	var logicalName = field.SchemaName!.ToLowerInvariant();	Reads and normalizes the field schema name.	Dataverse logical names are lowercase; ToLowerInvariant improves comparison reliability.	Supports idempotency checks.	Incorrect normalization may miss existing fields.
105	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
106	if (existingAttributes.Contains(logicalName))	Checks if the column already exists.	Avoids duplicate column creation attempts.	Allows safe rerun after partial success.	Without it, reruns fail on already-created fields.
107	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
108	Console.ForegroundColor = ConsoleColor.Yellow;	Changes terminal output color.	Red/yellow/green output makes failures, skips, and success visible.	Improves operator confidence during execution.	Not functionally required, but helps avoid missing errors.
109	Console.WriteLine(\$"SKIP existing: {logicalName}");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
110	Console.ResetColor();	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
111	continue;	Moves to the next field.	Used after skipping an already existing column.	Prevents executing the create request for that field.	Missing continue could attempt duplicate creation anyway.
112	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
113	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
114	try	Starts protected execution.	Any exception inside the block is caught and displayed.	Keeps the operator informed about exactly which field failed.	Without try/catch, the script may stop abruptly without clean context.
115	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
116	Console.WriteLine(\$"Creating: {logicalName} ...");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
117	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
118	var request = new CreateAttributeRequest	Creates a column-creation request.	The SDK uses this request type to add an AttributeMetadata column to a table.	This is the core Dataverse metadata operation for fields.	Wrong EntityName, Attribute, or SolutionUniqueName causes failures or wrong placement.
119	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
120	EntityName = EntityLogicalName,	Targets the table for a column request.	Uses EntityLogicalName, which is also journey participation ledger.	Ensures the column is created on the ledger table.	Wrong table name creates columns elsewhere or fails.
121	Attribute = field,	Passes the current field metadata to Dataverse.	The field variable contains the specific column definition from the list.	Creates the intended data type, label, description, and settings.	If wrong, the created column type may not match the data contract.
122	SolutionUniqueName = SolutionUniqueName	Associates created metadata with ASO.Core.	Dataverse stores the column as part of the target unmanaged solution.	Important for export, backup, managed deployment, and ALM.	Wrong solution name can create solution-layer confusion.
123	};	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
124	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
125	service.Execute(request);	Executes the current metadata request.	This is where Dataverse actually creates the current column.	Without this line, definitions exist only in memory.	Errors here identify invalid metadata or permissions.
126	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
127	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output color.	Red/yellow/green output makes failures, skips, and success visible.	Improves operator confidence during execution.	Not functionally required, but helps avoid missing errors.
128	Console.WriteLine(\$"CREATED: {logicalName}");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
129	Console.ResetColor();	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
130	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
131	catch (Exception ex)	Catches errors from a failed create attempt.	The exception object contains the SDK or Dataverse error message.	Allows troubleshooting without hiding the failure reason.	Catching without logging would conceal issues.
132	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
133	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output color.	Red/yellow/green output makes failures, skips, and success visible.	Improves operator confidence during execution.	Not functionally required, but helps avoid missing errors.
134	Console.WriteLine(\$"FAILED: {logicalName}");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
135	Console.WriteLine(ex.Message);	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
136	Console.ResetColor();	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
137	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
138	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
139	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
140	Console.WriteLine();	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
141	Console.WriteLine("Publishing Journey Participation Ledger table customizations...");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
142	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
143	service.Execute(new PublishXmlRequest	Publishes the table customization.	Dataverse metadata can be created but not fully available to the UI until published.	Ensures the new table and columns are visible and usable in Power Apps/Dynamics.	Publishing the wrong entity or skipping publishing delays visibility.
144	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
145	ParameterXml = "<importexportxml><entities><entity>aso_journey_participationledger</entity></entities></importexportxml>"	Publishes the table customization.	Dataverse metadata can be created but not fully available to the UI until published.	Ensures the new table and columns are visible and usable in Power Apps/Dynamics.	Publishing the wrong entity or skipping publishing delays visibility.

146	});	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
147	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
148	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output color.	Red/yellow/green output makes failures, skips, and success visible.	Improves operator confidence during execution.	Not functionally required, but helps avoid missing errors.
149	Console.WriteLine("Done. Validate in ASO.Core → Tables → Journey Participation Ledger → Columns.");	Writes progress information to the terminal.	The operator can see environment, solution, table, current field, and completion status.	Creates an execution trace useful for screenshots and handover.	Without output, troubleshooting partial runs is harder.
150	Console.ResetColor();	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
151	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
152	static bool EntityExists(ServiceClient service, string entityLogicalName)	Declares helper function to check table existence.	The function attempts to retrieve table metadata and returns true/false.	Prevents duplicate table creation on rerun.	Wrong implementation could falsely create or skip a table.
153	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
154	try	Starts protected execution.	Any exception inside the block is caught and displayed.	Keeps the operator informed about exactly which field failed.	Without try/catch, the script may stop abruptly without clean context.
155	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
156	service.Execute(new RetrieveEntityRequest	Creates a metadata retrieval request.	Retrieves entity or attribute metadata from Dataverse.	Used to check existing table/columns before writing changes.	Wrong filters can return too little or too much metadata.
157	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
158	LogicalName = entityLogicalName,	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
159	EntityFilters = EntityFilters.Entity,	Controls which metadata is retrieved.	Entity retrieves table-level information; Attributes retrieves column-level information.	Improves performance and accuracy.	Wrong filter can break existence checks.
160	RetrieveAsIfPublished = true	Reads metadata including unpublished changes.	Useful immediately after creating metadata or during iterative development.	Helps avoid false negatives when changes are not fully published yet.	If false, recently created items may not appear in checks.
161	});	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
162	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
163	return true;	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
164	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
165	catch	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
166	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
167	return false;	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
168	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
169	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
170	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
171	static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)	Declares helper function to retrieve existing columns.	Returns a case-insensitive set of current attribute logical names.	Provides fast and reliable duplicate checks.	If wrong, idempotency breaks.
172	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
173	var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest	Creates a metadata retrieval request.	Retrieves entity or attribute metadata from Dataverse.	Used to check existing table/columns before writing changes.	Wrong filters can return too little or too much metadata.
174	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
175	LogicalName = entityLogicalName,	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
176	EntityFilters = EntityFilters.Attributes,	Controls which metadata is retrieved.	Entity retrieves table-level information; Attributes retrieves column-level information.	Improves performance and accuracy.	Wrong filter can break existence checks.
177	RetrieveAsIfPublished = true	Reads metadata including unpublished changes.	Useful immediately after creating metadata or during iterative development.	Helps avoid false negatives when changes are not fully published yet.	If false, recently created items may not appear in checks.
178	});	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
179	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
180	return response.EntityMetadata.Attributes	Defines table metadata.	Specifies schema name, display labels, description, and ownership behavior.	Controls how the new custom table appears and behaves in Dataverse.	Wrong ownership or schema name is difficult to change later.
181	.Where(a => ! string.IsNullOrEmpty(a.LogicalName))	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
182	.Select(a => a.LogicalName!)	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
183	.ToHashSet(StringComparer.OrdinalIgnoreCase);	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
184	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
185	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
186	static Label Label(string value) => new(value, LanguageCode);	Defines the language code used for labels.	1031 is German and was used because the Phoenicix-CI organization uses German UI/base language behavior for some metadata labels.	Labels and descriptions must use a language enabled in the organization.	Invalid language code errors can occur if 1033/1031 does not match enabled languages.
187	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
188	static AttributeRequiredLevelManagedProperty Optional()	Defines optional-required-level helper.	Most ASO columns are optional because data will be filled later by integrations/automation.	Prevents blocking record creation during MVP.	Making fields required too early breaks standard Sales processes.

189	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
190	return new AttributeRequiredLevelManagedProperty(Attribut eRequiredLevel.None);	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
191	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
192	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
193	static StringAttributeMetadata Text(string schemaName, string displayName, int maxLength, string description)	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
194	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
195	return new StringAttributeMetadata	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
196	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
197	SchemaName = schemaName,	Sets a schema name.	Dataverse uses schema/logical names for ALM, SDK, API, integrations, and automation.	The aso_ naming keeps ASO components clearly separated.	Typos become permanent technical debt.
198	DisplayName = Label(displayName),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
199	Description = Label(description),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
200	RequiredLevel = Optional(),	Defines optional-required-level helper.	Most ASO columns are optional because data will be filled later by integrations/automation.	Prevents blocking record creation during MVP.	Making fields required too early breaks standard Sales processes.
201	MaxLength = maxLength	Sets maximum text length.	Dataverse requires length definitions for text/memo fields.	Prevents uncontrolled storage and aligns to expected payload size.	Too short truncates data; too long can be unnecessary.
202	};	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
203	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
204	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
205	static MemoAttributeMetadata Memo(string schemaName, string displayName, string description)	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
206	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
207	return new MemoAttributeMetadata	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
208	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
209	SchemaName = schemaName,	Sets a schema name.	Dataverse uses schema/logical names for ALM, SDK, API, integrations, and automation.	The aso_ naming keeps ASO components clearly separated.	Typos become permanent technical debt.
210	DisplayName = Label(displayName),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
211	Description = Label(description),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
212	RequiredLevel = Optional(),	Defines optional-required-level helper.	Most ASO columns are optional because data will be filled later by integrations/automation.	Prevents blocking record creation during MVP.	Making fields required too early breaks standard Sales processes.
213	MaxLength = 4000	Sets maximum text length.	Dataverse requires length definitions for text/memo fields.	Prevents uncontrolled storage and aligns to expected payload size.	Too short truncates data; too long can be unnecessary.
214	};	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
215	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
216	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
217	static DateTimeAttributeMetadata DateTimeField(string schemaName, string displayName, string description)	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
218	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
219	return new DateTimeAttributeMetadata	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
220	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
221	SchemaName = schemaName,	Sets a schema name.	Dataverse uses schema/logical names for ALM, SDK, API, integrations, and automation.	The aso_ naming keeps ASO components clearly separated.	Typos become permanent technical debt.
222	DisplayName = Label(displayName),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
223	Description = Label(description),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
224	RequiredLevel = Optional(),	Defines optional-required-level helper.	Most ASO columns are optional because data will be filled later by integrations/automation.	Prevents blocking record creation during MVP.	Making fields required too early breaks standard Sales processes.
225	Format = DateTimeFormat.DateAndTime,	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
226	DateTimeBehavior = DateTimeBehavior.UserLocal	Sets date/time behavior.	UserLocal shows timestamps adjusted for the viewer's time zone.	Appropriate for operational timestamps in the maker/user UI.	Wrong behavior can confuse users about event timing.
227	};	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
228	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
229	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
230	static PicklistAttributeMetadata LocalChoice(string schemaName, string displayName, string[] values, string description)	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
231	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.

232	var optionSet = new OptionSetMetadata	Configures choice/option-set metadata.	Controls whether options are local or global and adds selectable values.	Important for consistent reporting and controlled statuses.	Wrong option set design can require later refactoring.
233	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
234	IsGlobal = false,	Configures choice/option-set metadata.	Controls whether options are local or global and adds selectable values.	Important for consistent reporting and controlled statuses.	Wrong option set design can require later refactoring.
235	OptionSetType = OptionSetType.Picklist	Configures choice/option-set metadata.	Controls whether options are local or global and adds selectable values.	Important for consistent reporting and controlled statuses.	Wrong option set design can require later refactoring.
236	};	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
237	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
238	foreach (var value in values)	Starts the creation loop.	The script iterates through every planned field metadata definition.	Automates repetitive column creation safely.	Loop errors can stop or skip field creation.
239	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
240	optionSet.Options.Add(new OptionMetadata(Label(value), null));	Configures choice/option-set metadata.	Controls whether options are local or global and adds selectable values.	Important for consistent reporting and controlled statuses.	Wrong option set design can require later refactoring.
241	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
242	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
243	return new PicklistAttributeMetadata	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
244	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
245	SchemaName = schemaName,	Sets a schema name.	Dataverse uses schema/logical names for ALM, SDK, API, integrations, and automation.	The aso_ naming keeps ASO components clearly separated.	Typos become permanent technical debt.
246	DisplayName = Label(displayName),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
247	Description = Label(description),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
248	RequiredLevel = Optional(),	Defines optional-required-level helper.	Most ASO columns are optional because data will be filled later by integrations/automation.	Prevents blocking record creation during MVP.	Making fields required too early breaks standard Sales processes.
249	OptionSet = optionSet	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
250	};	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
251	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
252	(blank)	Blank spacer line.	No runtime effect; used to separate logical sections.	Improves readability during code review.	None; removing it only makes the file harder to read.
253	static PicklistAttributeMetadata GlobalChoice(string schemaName, string displayName, string globalChoiceName, string description)	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
254	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
255	return new PicklistAttributeMetadata	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
256	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
257	SchemaName = schemaName,	Sets a schema name.	Dataverse uses schema/logical names for ALM, SDK, API, integrations, and automation.	The aso_ naming keeps ASO components clearly separated.	Typos become permanent technical debt.
258	DisplayName = Label(displayName),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
259	Description = Label(description),	Sets human-readable metadata labels/descriptions.	These values are shown in Power Apps and help admins understand the component purpose.	Improves maintainability and handover quality.	Wrong labels confuse makers and business users.
260	RequiredLevel = Optional(),	Defines optional-required-level helper.	Most ASO columns are optional because data will be filled later by integrations/automation.	Prevents blocking record creation during MVP.	Making fields required too early breaks standard Sales processes.
261	OptionSet = new OptionSetMetadata	Configures choice/option-set metadata.	Controls whether options are local or global and adds selectable values.	Important for consistent reporting and controlled statuses.	Wrong option set design can require later refactoring.
262	{	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
263	IsGlobal = true,	Configures choice/option-set metadata.	Controls whether options are local or global and adds selectable values.	Important for consistent reporting and controlled statuses.	Wrong option set design can require later refactoring.
264	Name = globalChoiceName,	Executes part of the C# script.	This line participates in defining metadata, controlling flow, or running SDK operations.	It contributes to the repeatable Dataverse schema creation process.	Changing it without review can alter schema, execution, or error handling.
265	OptionSetType = OptionSetType.Picklist	Configures choice/option-set metadata.	Controls whether options are local or global and adds selectable values.	Important for consistent reporting and controlled statuses.	Wrong option set design can require later refactoring.
266	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
267	};	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.
268	}	Opens or closes a C# block/initializer.	Braces and terminators define the structure of lists, objects, loops, methods, and statements.	The compiler relies on these characters to understand scope and statement boundaries.	Missing or misplaced braces/semicolons cause build errors.

15. Appendix A - Full script

```
using Microsoft.Crm.Sdk.Messages;
using Microsoft.PowerPlatform.Dataverse.Client;
using Microsoft.Xrm.Sdk;
using Microsoft.Xrm.Sdk.Messages;
using Microsoft.Xrm.Sdk.Metadata;
```

```
const string DataverseUrl = "https://phoenicarix-ci.crm4.dynamics.com";
const string SolutionUniqueName = "ASOCore";
const string EntityLogicalName = "aso_journeyparticipationledger";
const int LanguageCode = 1031;
```

```
var connectionString =
    $"AuthType=OAuth;
    Url={DataverseUrl};
    LoginPrompt=Auto;
    ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;
    RedirectUri=http://localhost";
```

```
using var service = new ServiceClient(connectionString);
```

```
if (!service.IsReady)
{
    Console.ForegroundColor = ConsoleColor.Red;
    Console.WriteLine("Connection failed.");
    Console.WriteLine(service.LastError);
    Console.ResetColor();
    return;
}
```

```
Console.WriteLine($"Connected to: {DataverseUrl}");
Console.WriteLine($"Target solution: {SolutionUniqueName}");
Console.WriteLine($"Target table: {EntityLogicalName}");
Console.WriteLine();
```

```
if (!EntityExists(service, EntityLogicalName))
{
    Console.WriteLine("Creating Journey Participation Ledger table...");
```

```
var createEntityRequest = new CreateEntityRequest
{
    Entity = new EntityMetadata
    {
        SchemaName = "aso_journeyparticipationledger",
        DisplayName = Label("Journey Participation Ledger"),
        DisplayCollectionName = Label("Journey Participation Ledgers"),
        Description = Label("Records journey participation and interaction state once the Customer Insights phase is implemented."),
        OwnershipType = OwnershipTypes.OrganizationOwned
    },
    PrimaryAttribute = new StringAttributeMetadata
    {
        SchemaName = "aso_name",
        DisplayName = Label("Name"),
        Description = Label("Primary name for the Journey Participation Ledger record."),
        RequiredLevel = new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.ApplicationRequired),
        MaxLength = 200
    },
    SolutionUniqueName = SolutionUniqueName
};
```

```
service.Execute(createEntityRequest);
```

```

    Console.ForegroundColor = ConsoleColor.Green;
    Console.WriteLine("CREATED TABLE: aso_journeyparticipationledger");
    Console.ResetColor();
}
else
{
    Console.ForegroundColor = ConsoleColor.Yellow;
    Console.WriteLine("SKIP existing table: aso_journeyparticipationledger");
    Console.ResetColor();
}

var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);

var fields = new List<AttributeMetadata>
{
    Text("aso_recordtype", "Record Type", 100, "Business or Dataverse record type, for example Lead, Opportunity, Account, or Contact."),
    Text("aso_recordid", "Record ID", 100, "Dataverse row identifier of the related business record."),

    GlobalChoice("aso_lifecyclecommunicationstage", "Lifecycle Communication Stage", "aso_lifecyclecommunicationstage", "Lifecycle communication stage reused from the global ASO choice."),
    Text("aso_customerinsightsjourneyid", "Customer Insights Journey ID", 100, "Customer Insights journey identifier."),
    Text("aso_customerinsightsjourneyname", "Customer Insights Journey Name", 200, "Customer Insights journey name."),

    GlobalChoice("aso_participationstatus", "Participation Status", "aso_journeyparticipationstatus", "Journey participation status reused from the global ASO choice."),

    Text("aso_entrytriggername", "Entry Trigger Name", 200, "Name of the journey entry trigger, segment, custom action, or manual operation."),
    LocalChoice("aso_entrysource", "Entry Source",
        new[] { "DataverseTrigger", "Segment", "CustomAction", "ManualOps" },
        "Source that caused or will later explain the journey entry."),

    DateTimeField("aso_startedon", "Started On", "Timestamp when the journey participation started."),
    DateTimeField("aso_lastinteractionon", "Last Interaction On", "Timestamp of the latest known interaction."),
    LocalChoice("aso_lastinteractiontype", "Last Interaction Type",
        new[] { "EmailSent", "Open", "Click", "FormSubmit", "Reply", "Unsubscribe", "Bounce", "CustomAction" },
        "Latest tracked Customer Insights interaction type."),

    Text("aso_exitreason", "Exit Reason", 500, "Reason why the record exited or was removed from the journey."),
    Text("aso_correlationid", "Correlation ID", 100, "Correlation ID shared across Dataverse, Power Automate, Customer Insights, and orchestration components."),
    Memo("aso_errormessage", "Error Message", "Error details if journey participation processing fails.")
};

foreach (var field in fields)
{
    var logicalName = field.SchemaName!.ToLowerInvariant();

    if (existingAttributes.Contains(logicalName))
    {
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine($"SKIP existing: {logicalName}");
        Console.ResetColor();
        continue;
    }

    try
    {

```

```

        Console.WriteLine($"Creating: {logicalName} ...");

        var request = new CreateAttributeRequest
        {
            EntityName = EntityLogicalName,
            Attribute = field,
            SolutionUniqueName = SolutionUniqueName
        };

        service.Execute(request);

        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine($"CREATED: {logicalName}");
        Console.ResetColor();
    }
    catch (Exception ex)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine($"FAILED: {logicalName}");
        Console.WriteLine(ex.Message);
        Console.ResetColor();
    }
}

Console.WriteLine();
Console.WriteLine("Publishing Journey Participation Ledger table customizations...");

service.Execute(new PublishXmlRequest
{
    ParameterXml = "<importexportxml><entities><entity>aso_journeyparticipationledger</entity></entities></importexportxml>"
});

Console.ForegroundColor = ConsoleColor.Green;
Console.WriteLine("Done. Validate in ASO.Core → Tables → Journey Participation Ledger → Columns.");
Console.ResetColor();

static bool EntityExists(ServiceClient service, string entityLogicalName)
{
    try
    {
        service.Execute(new RetrieveEntityRequest
        {
            LogicalName = entityLogicalName,
            EntityFilters = EntityFilters.Entity,
            RetrieveAsIfPublished = true
        });

        return true;
    }
    catch
    {
        return false;
    }
}

```

```

static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)
{
    var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest
    {
        LogicalName = entityLogicalName,
        EntityFilters = EntityFilters.Attributes,
        RetrieveAsIfPublished = true
    });

    return response.EntityMetadata.Attributes
        .Where(a => !string.IsNullOrEmpty(a.LogicalName))
        .Select(a => a.LogicalName!)
        .ToHashSet(StringComparer.OrdinalIgnoreCase);
}

static Label Label(string value) => new(value, LanguageCode);

static AttributeRequiredLevelManagedProperty Optional()
{
    return new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.None);
}

static StringAttributeMetadata Text(string schemaName, string displayName, int maxLength, string description)
{
    return new StringAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MaxLength = maxLength
    };
}

static MemoAttributeMetadata Memo(string schemaName, string displayName, string description)
{
    return new MemoAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MaxLength = 4000
    };
}

static DateTimeAttributeMetadata DateTimeField(string schemaName, string displayName, string description)
{
    return new DateTimeAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        Format = DateTimeFormat.DateAndTime,
        DateTimeBehavior = DateTimeBehavior.UserLocal
    }
}

```

```

    };
}

static PicklistAttributeMetadata LocalChoice(string schemaName, string displayName, string[] values, string description)
{
    var optionSet = new OptionSetMetadata
    {
        IsGlobal = false,
        OptionSetType = OptionSetType.Picklist
    };

    foreach (var value in values)
    {
        optionSet.Options.Add(new OptionMetadata(Label(value), null));
    }

    return new PicklistAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        OptionSet = optionSet
    };
}

static PicklistAttributeMetadata GlobalChoice(string schemaName, string displayName, string globalChoiceName, string description)
{
    return new PicklistAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        OptionSet = new OptionSetMetadata
        {
            IsGlobal = true,
            Name = globalChoiceName,
            OptionSetType = OptionSetType.Picklist
        }
    };
}

```

16. Backup naming

Recommended backup names after successful validation:

ASO.Core_Phase2_CI_JourneyParticipationLedgerCreated_unmanaged_v1_0_20260524.zip

ASO.Core_Phase2_CI_JourneyParticipationLedgerCreated_managed_v1_0_20260524.zip